Week 5: Clustering and PCA — Online Retail Dataset

Table of contents

Step 1 — Unit of Analysis . . . 2
- What are we clustering? . . . 2
- Why customers? . . . 2

Step 2 — Data Loading and Feature Construction . . . 2
- Loading and Cleaning . . . 2
- Constructing Customer-Level Features . . . 3

Step 3 — Scaling . . . 5
- Why scaling is necessary . . . 5
- What method we used . . . 6
- What would happen without scaling . . . 7

Step 4 — Apply Clustering . . . 7
- 4.1 K-Means Clustering . . . 7
- 4.2 DBSCAN (Density-Based Clustering) . . . 11
- 4.3 Hierarchical Clustering . . . 13

Step 5 - Visualisation and Evaluation . . . 14
- PCA Visualisation . . . 14
- PCA Loadings . . . 16

Discussion and Limitations . . . 17
- What distinguishes each cluster? . . . 17
- Are clusters balanced? . . . 17
- Sensitivity to parameter choices . . . 18
- Limitations . . . 18
- Key takeaway . . . 18

Step 1 — Unit of Analysis

What are we clustering?

We will group **customers**. Every row in our data set corresponds to one particular **customer**, and is uniquely represented through the `Customer ID` column.

Why customers?

The Online Retail data has raw row level detail of each line item on an invoice that are too small to identify behavioral patterns in the individual events of each purchase.

Clustering at the **customer level** has value due to:

- The ability to identify **behavioural segments**, or groups of customers with common shopping behaviours.
- Segmentation is **actionable by the customer segment** when it comes to marketing, retention and inventory decision making.
- When I **aggregate all transactions per customer,** I produce a rich set of numeric features (spend, frequency, recency) to measure how customers have shopped over time.

Clustering by invoice or product can provide different insights into the behavior of customers but will have less business relevance than analysing at the customer level which aligns with how most retailers perform their analyses (i.e. RFM).

Step 2 — Data Loading and Feature Construction

Loading and Cleaning

```r
library(tidyverse)
library(dplyr)
library(lubridate)

# Loading all five CSV parts and combine
retail <- read.csv("online_retail_II.csv", header = TRUE)

cat("Raw dataset:", nrow(retail), "rows,", ncol(retail), "columns\n")
```

```
Raw dataset: 525461 rows, 8 columns
```

I clean the data by removing rows without a customer ID (cannot assign to a customer), negative quantities (returns/cancellations), and zero or negative prices (adjustments or errors):

```
retail <- retail %>%
  filter(!is.na(Customer.ID),
         Quantity > 0,
         Price > 0) %>%
  mutate(
    Customer.ID = as.integer(Customer.ID),
    InvoiceDate = dmy_hm(InvoiceDate),
    TotalSpend  = Quantity * Price
  )

cat("Cleaned dataset:", nrow(retail), "rows\n")
```

```
Cleaned dataset: 407664 rows
```

```
cat("Date range:", as.character(min(retail$InvoiceDate, na.rm = TRUE)),
    "to", as.character(max(retail$InvoiceDate, na.rm = TRUE)), "\n")
```

```
Date range: 2009-12-01 07:45:00 to 2010-12-09 20:01:00
```

```
cat("Unique customers:", n_distinct(retail$Customer.ID), "\n")
```

```
Unique customers: 4312
```

Constructing Customer-Level Features

Beginning with the creation of six numerical attributes (features) that will represent different aspects of customer purchasing behavior, we create the following six feature columns:

1. Total spend: This is the total amount spent by the customer as determined by the sum of Quantity * Price from all transactions.
2. Number of transactions: This attribute determines the number of invoices issued to the customer and therefore represents how frequently the customer purchases from the business.
3. Recency: This is an attribute that indicates the time in days since the most recent transaction was made by the customer, based on the data set end date. Therefore this captures whether or not the customer has been active within the past few months.

4. Number of distinct products: This is an attribute that identifies the number of unique stock codes purchased by the customer. It is an indication of the breadth of the customers' buying habits.
5. Average basket value: The average basket value is the average of all spends per invoice. The average basket value may be an indicator of the typical order size that can be expected from the customer.
6. Average quantity per transaction: The average quantity per transaction represents the mean quantity of items purchased per line item. The average quantity per transaction may be an indicator of the bulk-buying tendency of the customer.

Each of these six features provides a unique dimension of the way in which the customer interacts with the business, providing a complete picture of the customer's relationship with the company.

```r
reference_date <- max(retail$InvoiceDate, na.rm = TRUE) + days(1)

customers <- retail %>%
  group_by(Customer.ID) %>%
  summarise(
    total_spend          = sum(TotalSpend),
    num_transactions     = n_distinct(Invoice),
    recency              = as.numeric(difftime(reference_date,
                                               max(InvoiceDate),
                                               units = "days")),
    num_distinct_products = n_distinct(StockCode),
    avg_qty_per_txn      = mean(Quantity),
    .groups = "drop"
  )

# Average basket value (mean invoice total per customer)
avg_basket <- retail %>%
  group_by(Customer.ID, Invoice) %>%
  summarise(invoice_total = sum(TotalSpend), .groups = "drop") %>%
  group_by(Customer.ID) %>%
  summarise(avg_basket_value = mean(invoice_total), .groups = "drop")

customers <- customers %>%
  left_join(avg_basket, by = "Customer.ID")

cat("Customer-level dataset:", nrow(customers), "customers,",
    ncol(customers), "columns\n")
```

```
Customer-level dataset: 4312 customers, 7 columns
```

```r
summary(customers)
```

```
  Customer.ID     total_spend        num_transactions     recency
 Min.   :12346   Min.   :     2.95   Min.   :  1.000   Min.   :  1.00
 1st Qu.:13882   1st Qu.:   307.99   1st Qu.:  1.000   1st Qu.: 18.37
 Median :15350   Median :   706.02   Median :  2.000   Median : 53.14
 Mean   :15349   Mean   :  2048.24   Mean   :  4.456   Mean   : 91.45
 3rd Qu.:16834   3rd Qu.:  1723.14   3rd Qu.:  5.000   3rd Qu.:136.42
 Max.   :18287   Max.   :349164.35   Max.   :205.000   Max.   :374.42
 num_distinct_products avg_qty_per_txn    avg_basket_value
 Min.   :   1.00       Min.   :   1.000   Min.   :    2.95
 1st Qu.:  17.00       1st Qu.:   5.268   1st Qu.:  182.09
 Median :  38.00       Median :   9.343   Median :  287.37
 Mean   :  63.65       Mean   :  21.472   Mean   :  378.32
 3rd Qu.:  79.25       3rd Qu.:  14.235   3rd Qu.:  423.58
 Max.   :1741.00       Max.   :4460.294   Max.   :11880.84
```

The summary statistics display considerable right skew for both product diversity and spending a relatively small group of customers spent significantly more than the remainder.

Step 3 — Scaling

Why scaling is necessary

Clustering algorithms like K-means use Euclidean distance to assign observations to clusters. Without scaling:

- `total_spend` ranges from ~3 to ~349,000
- `recency` ranges from 0 to ~374 days
- `num_transactions` ranges from 1 to ~290

Raw value comparisons will show that "total_spend" dominates completely since it is an order(s) of magnitude larger than "days_since_last_order". Therefore a £1,000 difference in spending would swamp a 100-day difference in recency even if both were equally useful as segmentation criteria.

What method we used

We applied **standardisation** or **z-score scaling**: we subtracted the mean and then divided by the standard deviation for each variable. By doing this, I scaled each feature into having mean = 0 and standard deviation = 1 which meant that each feature was contributing equally to the distance calculation.

```
feature_cols <- c("total_spend", "num_transactions", "recency",
                  "num_distinct_products", "avg_basket_value",
                  "avg_qty_per_txn")

X <- customers[, feature_cols]

X_scaled <- scale(X)

cat("Scaled feature means (should be ~0):\n")
```

```
Scaled feature means (should be ~0):
```

```
round(colMeans(X_scaled), 4)
```

```
          total_spend      num_transactions               recency
                    0                     0                     0
num_distinct_products      avg_basket_value       avg_qty_per_txn
                    0                     0                     0
```

```
cat("\nScaled feature std devs (should be ~1):\n")
```

```

Scaled feature std devs (should be ~1):
```

```
round(apply(X_scaled, 2, sd), 4)
```

```
          total_spend      num_transactions               recency
                    1                     1                     1
num_distinct_products      avg_basket_value       avg_qty_per_txn
                    1                     1                     1
```

What would happen without scaling

If I were to use K-means without scaling, it would essentially group based solely upon "total_spend" as this is much larger in magnitude compared to the other feature values; therefore I would have tiers in terms of money rather than multivariate behavioral groupings. The other dimensions such as "recency" and "product_diversity", which are also important when trying to understand how customers behave, would be ignored.

Step 4 — Apply Clustering

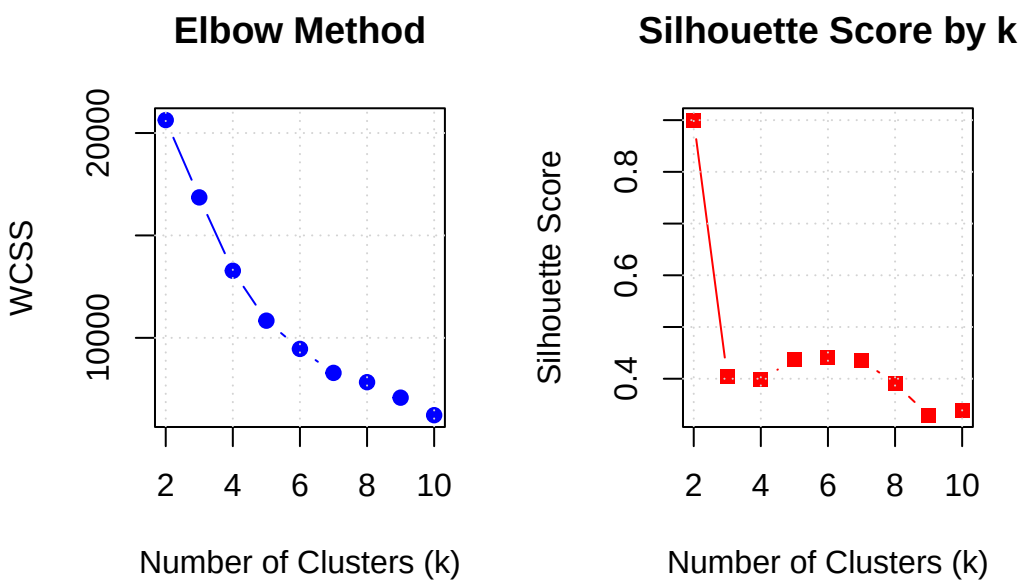
4.1 K-Means Clustering

Choosing k: Elbow Method and Silhouette Scores

To determine an appropriate number of clusters I ran K-means over k = 2 through 10 and plotted the WCSS and Silhouette score:

```
library(cluster)

K_range <- 2:10
wcss     <- numeric(length(K_range))
sil_vals <- numeric(length(K_range))

set.seed(42)
for (i in seq_along(K_range)) {
  k  <- K_range[i]
  km <- kmeans(X_scaled, centers = k, nstart = 20)
  wcss[i] <- km$tot.withinss
  sil_vals[i] <- mean(silhouette(km$cluster, dist(X_scaled))[, "sil_width"])
}

par(mfrow = c(1, 2))

plot(K_range, wcss, type = "b", pch = 19, col = "blue",
     xlab = "Number of Clusters (k)", ylab = "WCSS",
     main = "Elbow Method")
grid()

plot(K_range, sil_vals, type = "b", pch = 15, col = "red",
     xlab = "Number of Clusters (k)", ylab = "Silhouette Score",
```

```
     main = "Silhouette Score by k")
grid()
```




```
par(mfrow = c(1, 1))
```

The Elbow graph shows a steep decline in slope from k=2 to k=3 and then decreases gradually after that point. The Silhouette score is greatest at k=2; however, this will result in an oversimplified split. I choose **k=4** as a good balance of interpretability (I have enough clusters to identify meaningful segments), and parsimony (I do not have too many clusters to make them indistinguishable). The Silhouette Score at k=4 (~.40) represents a good amount of separation among the clusters, which is representative of most behavioral data.

Fitting K-Means with k = 4

```
set.seed(42)
kmeans_model <- kmeans(X_scaled, centers = 4, nstart = 20)

customers$kmeans_cluster <- kmeans_model$cluster

cat("K-Means Cluster Sizes:\n")
```

```
K-Means Cluster Sizes:
```

```r
table(customers$kmeans_cluster)
```

```
   1    2    3    4 
3208   12 1087    5 
```

Interpreting Cluster Centres

```r
cluster_means <- customers %>%
  group_by(kmeans_cluster) %>%
  summarise(across(all_of(feature_cols), mean), .groups = "drop")

cat("Cluster Centres (mean feature values):\n\n")
```

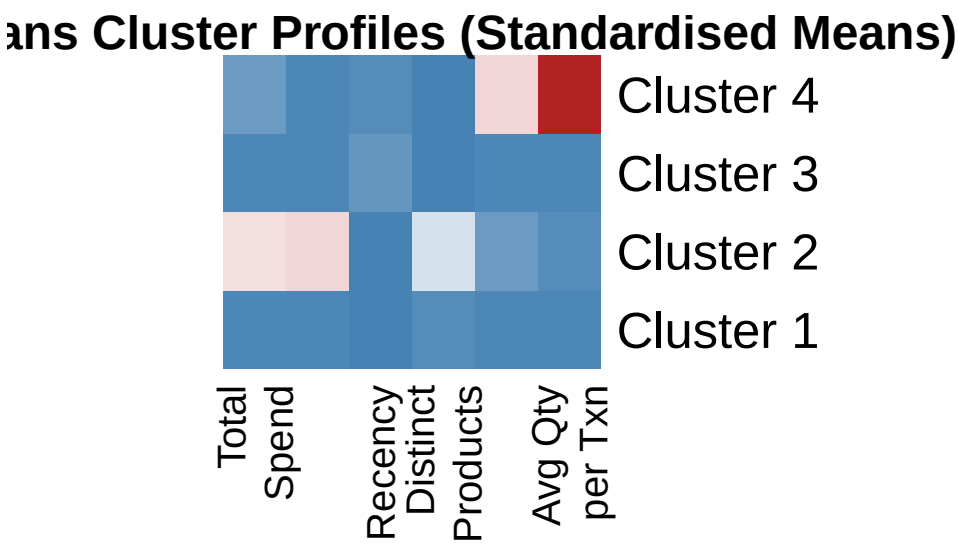
```
Cluster Centres (mean feature values):
```

```r
print(as.data.frame(cluster_means), digits = 2, row.names = FALSE)
```

```
 kmeans_cluster total_spend num_transactions recency num_distinct_products
              1        2118              5.0    41.9                    74
              2      116467            114.7     4.3                   800
              3         511              1.6   238.4                    25
              4       17140              3.2   133.3                    26
 avg_basket_value avg_qty_per_txn
              389              18
             1178              52
              307              19
             6962            2735
```

```r
# Heatmap of standardised cluster means
cluster_means_mat <- customers %>%
  group_by(kmeans_cluster) %>%
  summarise(across(all_of(feature_cols), mean), .groups = "drop") %>%
  select(-kmeans_cluster) %>%
  as.matrix()

# Standardise each column relative to overall mean/sd
```

```r
for (j in seq_len(ncol(cluster_means_mat))) {
  cluster_means_mat[, j] <- (cluster_means_mat[, j] - mean(X[[feature_cols[j]]])) /
                              sd(X[[feature_cols[j]]])
}

rownames(cluster_means_mat) <- paste("Cluster", 1:4)
colnames(cluster_means_mat) <- c("Total\nSpend", "Num\nTxns", "Recency",
                                 "Distinct\nProducts", "Avg Basket\nValue",
                                 "Avg Qty\nper Txn")

heatmap(cluster_means_mat,
        Colv = NA, Rowv = NA,
        scale = "none",
        col  = colorRampPalette(c("steelblue", "white", "firebrick"))(50),
        main = "K-Means Cluster Profiles (Standardised Means)",
        margins = c(10, 8))
```



Interpretation of the four clusters:

There are four segments based on the clusters (keep in mind that labels may be different when running this via Python, therefore you should be able to identify them by their profiles rather than the label):

- **Active/regular customers** (approximately ~3200): These customers have an average basket size that represents moderate spending. They also have some form of recent purchase history, as well as they have been purchasing a number of different products across approximately 5 purchases. This group represents the core of your customer base.
- **Infrequent/lapse customers** (approximately ~1090): Customers within this group have low spendings and very recent last purchases. In addition, they have few purchases, and they are only purchasing a limited number of products. These customers may be one time shoppers, or they may have churning issues.
- **VIP/Wholesale** (approximately ~12): The top tier of spenders, these customers have extremely high total spending amounts. In addition, they have extremely high numbers of transactions (averaging ~115 invoices) as well as they are purchasing a very wide variety of products (averaging ~800 unique products). These customers are likely resellers or a form of a key account.
- **Bulk Purchasers** (approximately ~5): Bulk purchasers have the highest averages for basket size, as well as the highest averages for quantity per transaction. However, they have the fewest amount of different transactions. Bulk purchasers purchase large quantities per order, however they do so rarely.

As you can see, there is a highly skewed distribution of clusters, with the majority of customers falling into the "regular" category. This is typical of most retail customer bases, which are characterised by a long tail distribution.

4.2 DBSCAN (Density-Based Clustering)

DBSCAN does not require specifying the number of clusters. Instead, it identifies dense regions and labels sparse points as noise. I test several `eps` values:

```r
library(dbscan)

cat("DBSCAN sensitivity to eps (minPts = 10):\n\n")
```

```
DBSCAN sensitivity to eps (minPts = 10):
```

```r
for (eps_val in c(0.8, 1.0, 1.5, 2.0, 2.5, 3.0)) {
  db <- dbscan(X_scaled, eps = eps_val, minPts = 10)
  n_clusters <- length(unique(db$cluster[db$cluster != 0]))
  n_noise    <- sum(db$cluster == 0)
```

```r
  cat(sprintf("  eps = %.1f: %d cluster(s), %d noise points (%.1f%%)\n",
              eps_val, n_clusters, n_noise, n_noise / nrow(X_scaled) * 100))
}
```

```
  eps = 0.8: 1 cluster(s), 183 noise points (4.2%)
  eps = 1.0: 1 cluster(s), 132 noise points (3.1%)
  eps = 1.5: 1 cluster(s), 77 noise points (1.8%)
  eps = 2.0: 1 cluster(s), 52 noise points (1.2%)
  eps = 2.5: 1 cluster(s), 39 noise points (0.9%)
  eps = 3.0: 1 cluster(s), 32 noise points (0.7%)
```

```r
# Apply DBSCAN with eps = 1.5
db_model <- dbscan(X_scaled, eps = 1.5, minPts = 10)
customers$dbscan_cluster <- db_model$cluster

cat("\nDBSCAN Cluster Sizes (eps = 1.5):\n")
```

```
DBSCAN Cluster Sizes (eps = 1.5):
```

```r
table(customers$dbscan_cluster)
```

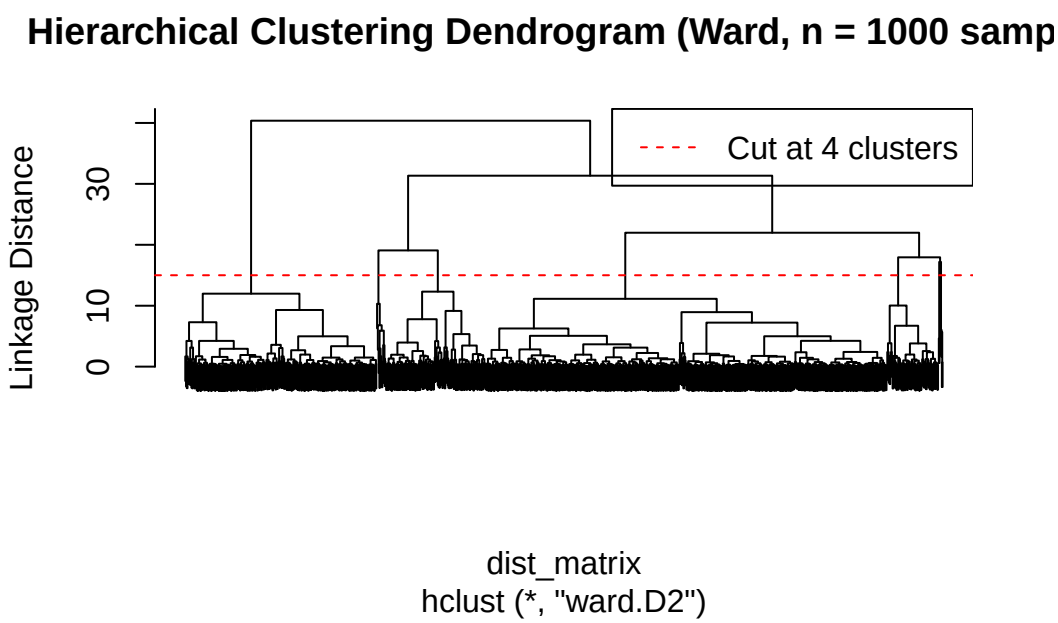
```
   0    1
  77 4235
```

Interpretation: The results from using DBSCAN show **a single dense-core cluster** in every case where a different **number of "noise" or outlier points are found** (cluster 0 in R's dbscan). What this means is that we can say that the customer data constitutes a single **continuous-density area in feature space**, with the extreme (outlier) customers located near the edge of this area. While DBSCAN effectively identifies the outlier (or "noise") customers (the points that represent the most extreme customers), it does **not identify two or more discrete, dense areas**. Thus, based upon these results, I have evidence that the data has a **single core with a long tail**, as opposed to **two or more separate clusters**.

4.3 Hierarchical Clustering

I apply hierarchical clustering, which creates a tree of merged subsets, by using Ward's Method (which uses the minimum increase in sum of squares at each step of merging; i.e., similar to K-means) to a randomly sampled set of 1000 customers so as to speed up computation:

```
set.seed(42)
sample_idx    <- sample(1:nrow(X_scaled), 1000)
X_sample      <- X_scaled[sample_idx, ]

dist_matrix   <- dist(X_sample)
hc_model      <- hclust(dist_matrix, method = "ward.D2")

plot(hc_model, labels = FALSE,
     main = "Hierarchical Clustering Dendrogram (Ward, n = 1000 sample)",
     ylab = "Linkage Distance")
abline(h = 15, col = "red", lty = 2)
legend("topright", legend = "Cut at 4 clusters", col = "red", lty = 2)
```




```
hc_clusters <- cutree(hc_model, k = 4)

cat("Hierarchical Cluster Sizes (k = 4, sample of 1000):\n")
```

```
Hierarchical Cluster Sizes (k = 4, sample of 1000):
```

```r
table(hc_clusters)
```

```
hc_clusters
  1   2   3   4
 74 530 143 253
```

The dendrogram indicates that there is a significant separation (large jumps) from the first two levels (top), which would suggest an initial grouping of 2-3 large groups. Further down in the tree (fine structure), there will be smaller subgroups. The final clustering at four groups has resulted in a distribution similar to that obtained by K-means; i.e., one large cluster, one moderate sized cluster and several very small clusters of extreme customers. The fact that these algorithms have produced a similar structure to the customer types increases confidence in the structure that I have identified.

Step 5 - Visualisation and Evaluation

PCA Visualisation

I use PCA to reduce our 6 features to 2 dimensions for plotting. PCA finds the directions of maximum variance in the data:

```r
pca_result <- prcomp(X_scaled, center = FALSE, scale. = FALSE)

var_explained <- summary(pca_result)$importance[2, 1:2]
cat("PC1 explains", round(var_explained[1] * 100, 1), "% of variance\n")
```

```
PC1 explains 39.5 % of variance
```

```r
cat("PC2 explains", round(var_explained[2] * 100, 1), "% of variance\n")
```

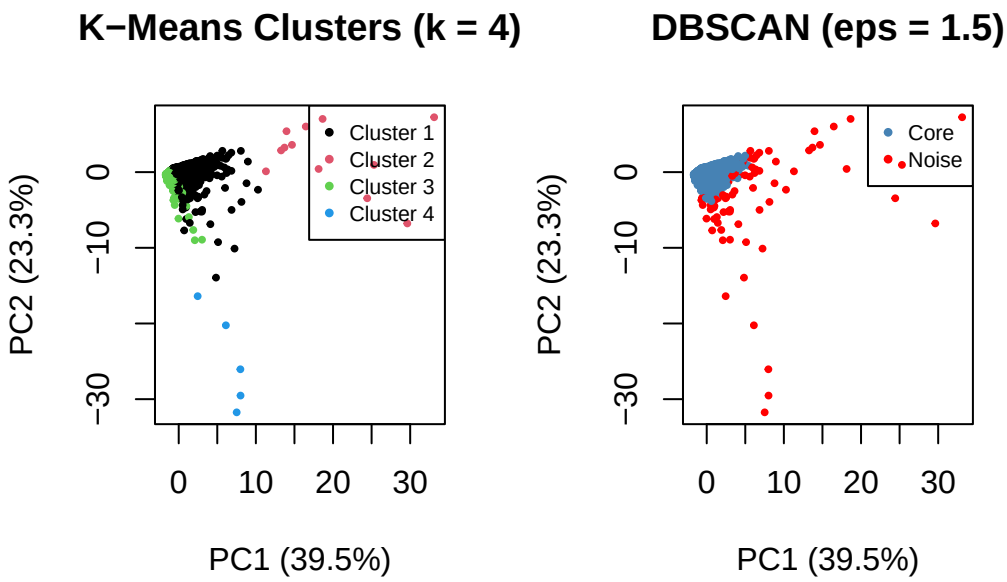
```
PC2 explains 23.3 % of variance
```

```r
cat("Total explained by 2 components:",
    round(sum(var_explained) * 100, 1), "%\n")
```

```
Total explained by 2 components: 62.8 %
```

```r
X_pca <- pca_result$x[, 1:2]
```

```r
par(mfrow = c(1, 2))

# K-Means PCA plot
plot(X_pca[, 1], X_pca[, 2],
     col  = customers$kmeans_cluster,
     pch  = 20, cex = 0.6,
     xlab = paste0("PC1 (", round(var_explained[1] * 100, 1), "%)"),
     ylab = paste0("PC2 (", round(var_explained[2] * 100, 1), "%)"),
     main = "K-Means Clusters (k = 4)")
legend("topright", legend = paste("Cluster", sort(unique(customers$kmeans_cluster))),
       col = sort(unique(customers$kmeans_cluster)), pch = 20, cex = 0.7)

# DBSCAN PCA plot
dbscan_colours <- ifelse(customers$dbscan_cluster == 0, "red", "steelblue")
plot(X_pca[, 1], X_pca[, 2],
     col  = dbscan_colours,
     pch  = 20, cex = 0.6,
     xlab = paste0("PC1 (", round(var_explained[1] * 100, 1), "%)"),
     ylab = paste0("PC2 (", round(var_explained[2] * 100, 1), "%)"),
     main = "DBSCAN (eps = 1.5)")
legend("topright", legend = c("Core", "Noise"),
       col = c("steelblue", "red"), pch = 20, cex = 0.7)
```




```
par(mfrow = c(1, 1))
```

A large percentage (~63%) of variance is explained by PCA projection, which reveals a tight concentration of the majority of customers at the origin with the exception of some outliers located on PC1. The K-means method divides the space into four areas. In contrast to K-means, the DBSCAN approach creates a central cluster of high-density customer data and labels all outliers in the database as "noise." Therefore, the two clustering approaches can be used together for the purpose of obtaining an overall view of the clustering results. The K-means approach allows for the identification of specific segments that can be interpreted. The DBSCAN approach provides a way to understand the general density structure of the data, and it also helps to identify those customer records that represent outliers.

PCA Loadings

```
loadings <- pca_result$rotation[, 1:2]
colnames(loadings) <- c("PC1", "PC2")

cat("PCA Loadings:\n\n")
```

```
PCA Loadings:
```

```r
print(round(loadings, 3))
```

```
                        PC1    PC2
total_spend           0.520 -0.102
num_transactions      0.566  0.184
recency              -0.265 -0.220
num_distinct_products 0.513  0.239
avg_basket_value      0.252 -0.620
avg_qty_per_txn       0.110 -0.682
```

The loadings for PC1 are primarily associated with spend-related characteristics (e.g., total spend, number of transactions, distinct products) and therefore represent an overall "customer activity" axis. The loadings for PC2 are related to recency and average quantity and thus distinguish recent active customers from those who are inactive.

Discussion and Limitations

What distinguishes each cluster?

Four customer groups emerged from a k-means analysis with distinct behavioral patterns:

1. **Regular Active Customers**: Most frequent buyers; moderate to recent purchase history; moderate total spend.
2. **Lapsed/Infrequent Buyers**: Low-spend; High Recency; do not buy frequently (or at all).
3. **VIP/Wholesale Buyers**: Extremely high total spend; extremely high total number of transactions; extreme purchasing power.
4. **Bulk Purchasers**: Unusually large quantity in each transaction; do not necessarily have an unusually high total number of transactions or spend.

Are clusters balanced?

No - nor should they be expected to. A natural power law exists for retail customer distributions most customers buy moderately (a long-tail of very-high value buyers) so it is to be expected that cluster sizes will have some imbalance in size due to how many people exist at each end of the spectrum.

Sensitivity to parameter choices

- **K in K-Means**: Changing k from 3 to 5 mostly splits the large cluster that was regular into smaller sections of the same type; The extreme cluster types have remained the same. This implies that extreme customer groups are strong and the border between "regular" and "lapsed" customers is less clear cut.
- **Eps in DBSCAN**: DBSCAN has identified exactly 1 core cluster no matter what value of eps I used. This implies there are not at least two or more distinct, separated, dense regions in this data set – it is a gradual distribution rather than discrete clusters.
- **Scaling Method**: I used Standardisation (z-score) for the scaling. Min-max Scaling will cause outliers to be compressed differently and could cause differences in how cluster boundaries are defined, especially with respect to the extreme segments.

Limitations

1. **Only 63% of total variance explained by two PCA components.** The cluster images present a compressed image; the cluster images may represent separation within unseen dimensionality.
2. **K-means assumes spherical clusters,** which does not describe the heavy tailed distribution of retail spending. The small extreme clusters have been at least partially created as a result of K-means assuming spherical clusters.
3. **Customer segment definition is based on feature selection.** Six features that place emphasis on both financial expenditure and frequency were selected to define similarities between customers. Different customer segments may be produced with alternative combinations of features (for example geographic diversity or purchasing patterns related to time of day).
4. **Cancellations and returns were excluded from the analysis, therefore we may be missing "problematic" customer segments that are defined by the number of returns.
5. **The clustering represents a snapshot of customer behaviour.** Customer behaviour can change over time; the customer segments described here represent behaviour across all of the data, rather than the current behaviour of our customers.

Key takeaway

Different algorithms offer a different look at the same information. K-means will give you segments of your data that can help guide your marketing strategy. DBSCAN shows us that we are dealing with one large segment of customers, but there are some extreme outliers in our customer base. The large scale of the groups (segments) is confirmed with hierarchical clustering. There isn't an algorithm that will give you the "right" or "true" answer to the question of how to segment your customers; each method has its own set of assumptions,

and it's only through interpreting the results of each segmentation method that they provide value.